



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.:2

Team No.:18

Project Title: Linux File Backup and Synchronization System Using System Calls.

Team Members

Roll Number	Student Name	Signature
2520039620	Dhruva Datta Vishnubhotla	
2520030331	Arnav Chitikena	
2520030114	Ashesh Charlapu	
2520030521	A Vishal Goud	

1. Abstract

In the current digital world, users and organizations generate and store large amounts of important data on computers and local storage devices. This data can be lost or become inaccessible due to accidental deletion, hardware failure, system crashes, unexpected power failures, malware attacks, and human errors. Manual backup methods are often time-consuming and may result in outdated, incomplete, or duplicate backup files. Repeatedly copying unchanged files also causes unnecessary disk usage and file input/output operations.

*The **Linux File Backup and Synchronization System Using System Calls** is proposed to address these issues by maintaining an updated backup of files from a source directory. The system traverses the source directory, checks file information, identifies new or modified files, and copies only the required files to the backup location. Unchanged files are skipped to reduce unnecessary operations.*

*The project is implemented in **C on Linux/Ubuntu** using Linux system calls and POSIX APIs. File operations are performed using `open()`, `read()`, `write()`, and `close()`, while `stat()` is used to obtain file metadata such as size and modification time. Directory traversal is handled using `opendir()`, `readdir()`, and `closedir()`, with `mkdir()` used to create required backup directories. Error handling is also incorporated to manage unsuccessful file and directory operations.*

*The expected outcome is a practical, medium-level backup and synchronization system that demonstrates **file management, directory handling, file descriptors, metadata management, low-level I/O, and system-call usage** in Operating Systems and Systems Programming.*

2. Problem Statement

In the current digital world, users and organizations continuously create and store large amounts of important data in computers and local storage systems. However, files can be lost or become unavailable due to accidental deletion, hardware failure, system crashes, malware or ransomware attacks, unexpected power failures, and human errors. Maintaining backups manually is also time-consuming and can result in outdated or incomplete backup copies. Another common issue is the presence of duplicate files and unnecessary copying of files that have not changed. Therefore, this project aims to develop a **Linux-based File Backup and Synchronization System** that automatically manages backup copies of files and keeps the backup directory synchronized with the source directory. The system identifies new or modified files and copies only the required data, reducing unnecessary file operations. The project uses Linux system calls and POSIX APIs for file handling, directory traversal, metadata access, and low-level input/output operations. This provides a practical demonstration of how Operating Systems manage files and resources through system-level interfaces.

3. Objectives

1. To develop a Linux-based file backup system that creates reliable backup copies of important files and directories.
2. To detect new and modified files using file metadata and synchronize only the required files with the backup directory.
3. To implement file and directory operations using Linux system calls and POSIX APIs, demonstrating low-level file handling and resource management.
4. To reduce unnecessary file copying and storage usage by identifying unchanged files and maintaining an updated, synchronized backup.

4. Proposed Methodology

The proposed system will be implemented as a Linux command-line application using C. The user provides a source directory containing the original files and a backup directory where synchronized copies are maintained.

The system first validates the given directories and traverses the source directory using Linux/POSIX directory APIs. For every file encountered, the system obtains information such as file size and modification time using `stat()`. This information is compared with the corresponding file in the backup directory to determine whether the file is new, modified, or unchanged. For new or modified files, the system opens the source and destination files using `open()`. File contents are transferred using `read()` and `write()`, and the files are properly closed using `close()`. If required backup directories do not exist, they are created using `mkdir()`. Directory contents are accessed using `opendir()`, `readdir()`, and `closedir()`.

5. Operating Systems Concepts / Linux APIs Use

OS Concept / Linux API / System Call	Purpose in the Project
<code>open()</code> / <code>close()</code>	Opens source and backup files and releases file descriptors after operations.
<code>read()</code> / <code>write()</code>	Reads data from source files and writes it to backup files during copying.
<code>stat()</code>	Retrieves file size and modification time to detect new or modified files.
<code>opendir()</code> / <code>readdir()</code> / <code>closedir()</code>	Traverses directories and identifies files and subdirectories for synchronization.
Error Handling (<code>errno</code> , return values)	Detects and handles errors during file access, reading, writing, directory operations, and other system calls.
<code>stat()</code>	Obtains file information such as size and modification time to detect changes.

6. Individual Contribution

<i>Roll Number</i>	<i>Student Name</i>	<i>Individual Responsibility</i>
2520039620	Dhruva Datta Vishnubhotla	Implement file backup operations using <code>open()</code> , <code>read()</code> , <code>write()</code> , and <code>close()</code> system calls, including basic file-copy functionality.
2520030114	Ashesh Charlapu	Implement file metadata checking using <code>stat()</code> and compare file properties such as size and modification time to identify changed files
2520030331	Arnav Chitikena	Implement the file synchronization logic , integrate all modules, handle system-call errors, and perform testing and debugging of the complete system.
2520030521	A Vishal Goud	Implement directory traversal using <code>opendir()</code> , <code>readdir()</code> , and <code>closedir()</code> , and handle creation of required directories using <code>mkdir()</code> .

7. Tools / Platforms / Software Used

<i>Tool / Platform / Software</i>	<i>Purpose</i>
Linux / Ubuntu	Operating environment for developing and executing the system-call based backup application.
C / C++	Used to implement the backup, synchronization, file handling, and system-call operations.
GCC Compiler	Used to compile and execute the C source code on Linux.
VS Code	Used for writing, editing, debugging, and managing the project source code.
Git / GitHub	Used for source-code version control, collaboration, and project repository management.
GDB	Used for debugging the C program and identifying runtime or logical errors.

8. Expected Outcome

(Describe the expected functionality, output, and outcome of the project.)

The project is expected to provide a functional **Linux-based file backup and synchronization utility** capable of maintaining an updated copy of files from a source directory in a backup directory. The system will identify new and modified files, copy the required files, update outdated backup copies, and skip unchanged files to reduce unnecessary file operations.

The system will also be capable of traversing directories, creating required backup directories, accessing file metadata, and performing low-level file operations using Linux system calls. Appropriate error messages and operation status will be displayed when file or directory operations succeed or fail.

*The final outcome will be a practical demonstration of **Operating System file management, system calls, file descriptors, directory handling, metadata management, and low-level I/O**. The project will provide a simple and efficient approach to reducing the risk of data loss caused by accidental deletion, hardware failure, system crashes, and other common file-storage problems.*

9. GitHub Repository

GitHub Repository Link: https://github.com/asheshchallarp19/2520030114-39620-331_OSSP

Repository Created:

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: _____

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____